RA2311047010044 38

INDEX

NAME K.D.D.SaiAbhisam SUBJECT Deep learning lab obde

STD. _____ DIV. _____ ROLL NO. _____ SCHOOL _____

SR. NO.	PAGE NO.	TITLE	DATE	TEACHER'S SIGN / REMARKS
1		Analyze and exploring the deep learning platforms	24-07-25	
2		Implement a classifier using open-source data set	7/8/25	
3.		Study of the classifiers with respect to statistical parameters	7/8/25	
4.		Build a simple feed forward neural network to recognize hand written characters	14/8/25	
5.		Study of activation function and its role	28-8-25	
6.		Implement gradient descent & back propagation in deep neural network.	09-9-25	
7.		Build a CNN model to classify Cats & dogs images	16-9-25	
8.		Experiment using LSTM		
9.		Build a RNN		
10		Perform Compression on MNIST data set using Autoencoder		
11.		to implement experiment using variational Autoencoder		
12.		Implement DCGAN to generate colour images	27/10	
13.		understanding the architecture of Pre-trained model	27/10	
14.		Pre-trained CNN model as feature extractor using TLM		
15.		Implement a YOLO model to detect objects		

Completed

11. to implement experiment using variational Autoencoder

Aim :- to implement the experiment using variational auto-encoder and generate the images.

Objective :-

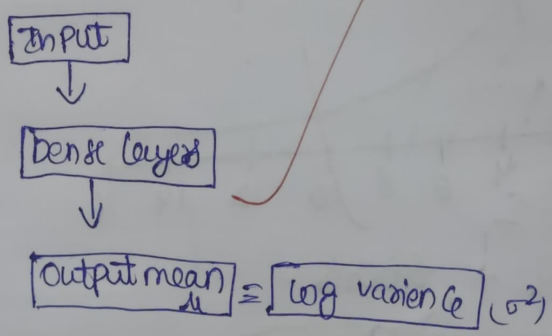
* to understand the concept of latent space representation
* to generate new data by sampling from the learned distribution.

Pseudo Code :-

(1) Import libraries like tensorflow

(2) Load and normalize MNIST dataset

(3) define encoder



(4) Reparameterization trick

$$z = \mu + \sigma * \varepsilon$$

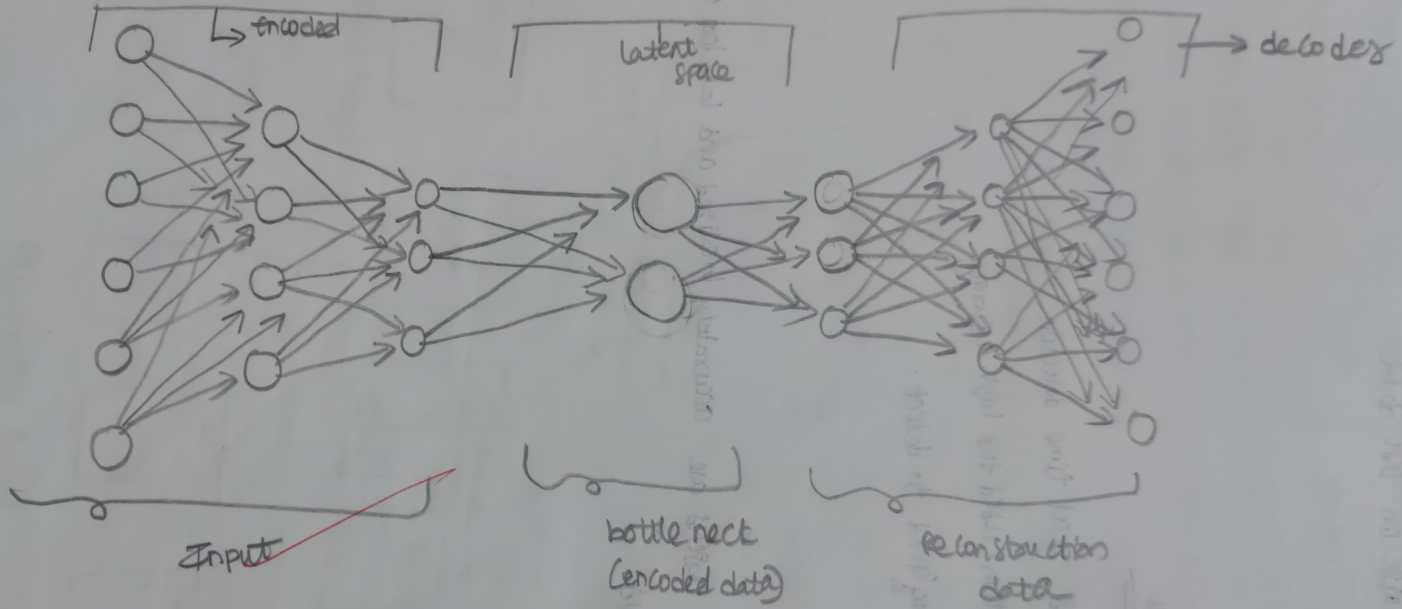
(5) Define decoder to reconstruct images from z

(6) Define loss = reconstruction_loss + KL_divergence

(7) Compile and train VAE model

(8) Generate new images from random latent vectors

Variational auto encoder architecture



```python
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms
from torch.utils.data import DataLoader
import matplotlib.pyplot as plt
import numpy as np

# 1. Device configuration
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

# 2. Load MNIST dataset (open dataset)
transform = transforms.Compose([
    transforms.ToTensor()
])

train_dataset = datasets.MNIST(root='./data', train=True, transform=transform, download=True)
test_dataset = datasets.MNIST(root='./data', train=False, transform=transform, download=True)

train_loader = DataLoader(train_dataset, batch_size=128, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=128, shuffle=False)

# 3. Define Variational Autoencoder
class VAE(nn.Module):
    def __init__(self, latent_dim=2):  # latent_dim=2 for easy visualization
        super(VAE, self).__init__()
        self.fc1 = nn.Linear(784, 400)
        self.fc_mu = nn.Linear(400, latent_dim)
        self.fc_logvar = nn.Linear(400, latent_dim)
        self.fc2 = nn.Linear(latent_dim, 400)
        self.fc3 = nn.Linear(400, 784)
        self.relu = nn.ReLU()
        self.sigmoid = nn.Sigmoid()

    def encode(self, x):
        h = self.relu(self.fc1(x))
        mu = self.fc_mu(h)
```

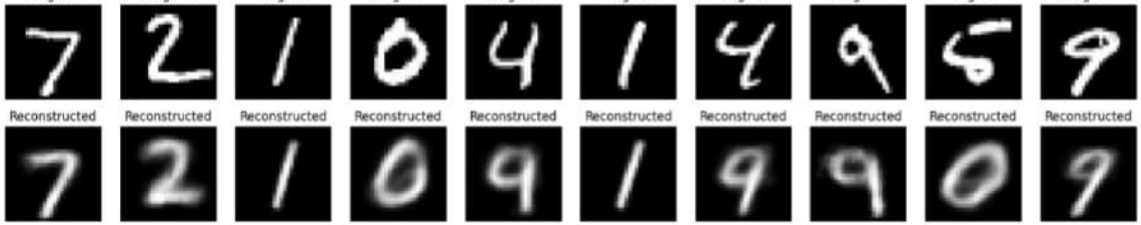
```
with torch.no_grad():
    for data, _ in test_loader:
        data = data.view(-1, 784).to(device)
        recon, mu, logvar = model(data)
        break

# Show original vs reconstructed
data = data.cpu().view(-1, 1, 28, 28)
recon = recon.cpu().view(-1, 1, 28, 28)

n = 10
plt.figure(figsize=(20, 4))
for i in range(n):
    # Original
    ax = plt.subplot(2, n, i + 1)
    plt.imshow(data[i].squeeze(), cmap='gray')
    plt.title("Original")
    plt.axis('off')
    # Reconstructed
    ax = plt.subplot(2, n, i + 1 + n)
    plt.imshow(recon[i].squeeze(), cmap='gray')
    plt.title("Reconstructed")
    plt.axis('off')
plt.show()

# 9. Visualize latent space (2D)
from sklearn.manifold import TSNE

model.eval()
z_list = []
labels_list = []

with torch.no_grad():
    for data, labels in test_loader:
        data = data.view(-1, 784).to(device)
        mu, logvar = model.encode(data)
        z = mu.cpu().numpy()
        z_list.append(z)
        labels_list.append(labels.numpy())
```

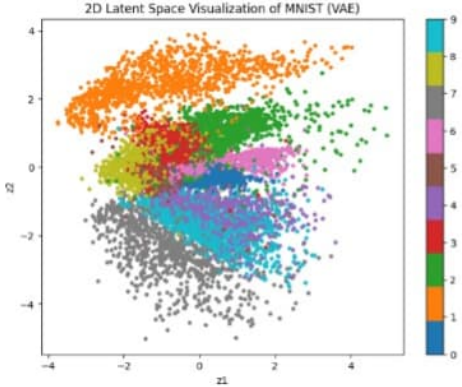
```
model.eval()
z_list = []
labels_list = []

with torch.no_grad():
    for data, labels in test_loader:
        data = data.view(-1, 784).to(device)
        mu, logvar = model.encode(data)
        z = mu.cpu().numpy()
        z_list.append(z)
        labels_list.append(labels.numpy())

z = np.concatenate(z_list)
labels = np.concatenate(labels_list)

plt.figure(figsize=(8, 6))
scatter = plt.scatter(z[:, 0], z[:, 1], c=labels, cmap='tab10', s=10)
plt.colorbar(scatter)
plt.title("2D Latent Space Visualization of MNIST (VAE)")
plt.xlabel("z1")
plt.ylabel("z2")
plt.show()
```

```
plt.show()
```

```
Epoch [1/15], Loss: 191.1015
Epoch [2/15], Loss: 168.2837
Epoch [3/15], Loss: 164.2876
Epoch [4/15], Loss: 162.2186
Epoch [5/15], Loss: 160.6326
Epoch [6/15], Loss: 159.3979
Epoch [7/15], Loss: 158.3026
Epoch [8/15], Loss: 157.2504
Epoch [9/15], Loss: 156.4238
Epoch [10/15], Loss: 155.5865
Epoch [11/15], Loss: 154.8330
Epoch [12/15], Loss: 154.1260
Epoch [13/15], Loss: 153.4800
Epoch [14/15], Loss: 152.8805
Epoch [15/15], Loss: 152.3461
```

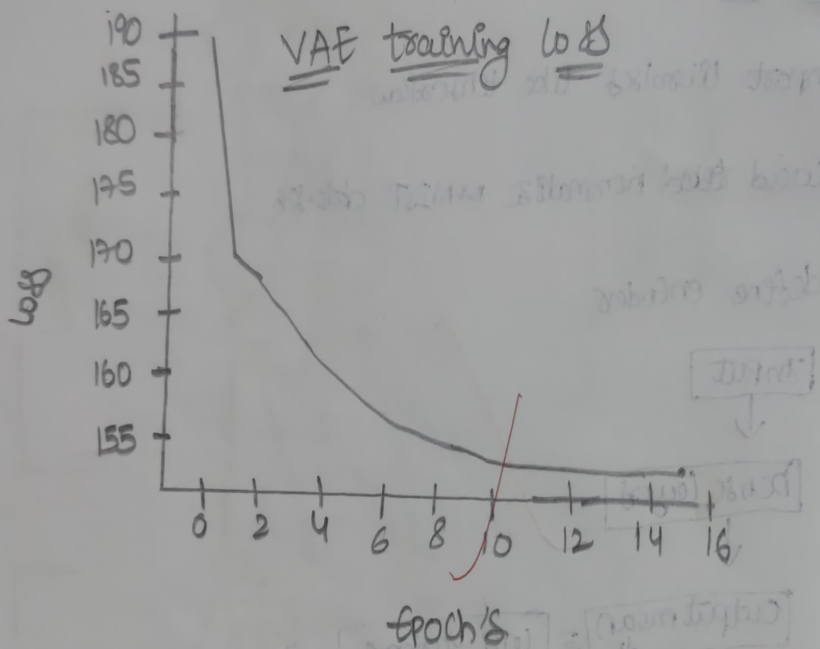
Epoch


Original
Reconstructed

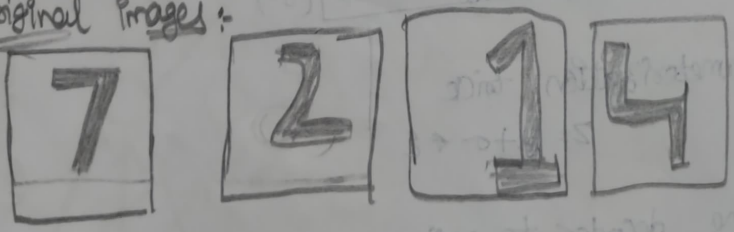

2D Latent Space Visualization of MNIST (VAE)
z1
z2

Output

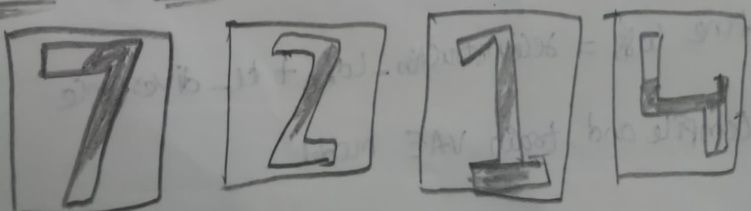
Epoch [1/15] Loss: 191.1015
Epoch [2/15] Loss: 168.2837
Epoch [3/15] Loss: 164.2876
Epoch [4/15] Loss: 162.2186
Epoch [5/15] Loss: 160.6326
Epoch [6/15] Loss: 159.3979
Epoch [7/15] Loss: 158.2504
Epoch [8/15] Loss: 157.2501
Epoch [9/15] Loss: 156.4234
Epoch [10/15] Loss: 155.5816
Epoch [11/15] Loss: 154.8334
Epoch [12/15] Loss: 154.1265
Epoch [13/15] Loss: 153.4805
Epoch [14/15] Loss: 152.8815
Epoch [15/15] Loss: 152.3145



original images:



Reconstruction images



Observation

* the variational autoencoder learns input images into a latent space and can generate smooth interpolations between digits.

Result :- Generated images resemble hand written digits, demonstrating the model's ability to learn data distribution effectively.

11/7/23